

Manual da MaestrIA

Este é o manual vivo da aplicação. Ele descreve o que a MaestrIA faz, como faz, quando cada parte é acionada e como operar os fluxos principais com exemplos.

Regra de manutenção: qualquer alteração que mude comportamento visível, comandos, configuração, runtime, integrações, UI, assets, segurança, recovery ou fluxo operacional deve atualizar este manual na mesma mudança. Quando este Markdown mudar, gere novamente `docs/MANUAL.pdf`. Quando uma alteração não exigir atualização aqui, registre isso no handoff da tarefa.

1. O Que É A MaestrIA

A MaestrIA é um escritório local-first para times de agentes de IA. Em vez de concentrar tudo em uma única conversa longa, ela organiza o trabalho em:

- canais compartilhados;
- DMs e sessões 1:1;
- tarefas com dono, status, bloqueios, revisões e entregas;
- runners novos por turno;
- broker local que acorda agentes sob demanda;
- ferramentas MCP com escopo por agente;
- worktrees Git isoladas por agente;
- interface web e TUI para acompanhamento humano.

O objetivo é permitir que o operador veja o trabalho acontecendo, responda quando for necessário e consiga auditar o que foi feito depois.

Exemplo de uso:

Voce abre o escritório, pede no canal geral:

```
"@pm transforme essa ideia em plano de execucao e chame engenharia quando estiver claro."
```

O broker registra a mensagem, identifica a menção ao PM, acorda o runner do agente e transmite a resposta de volta para o canal.

Se o PM criar tarefas, elas aparecem na inbox. Se alguma depender de você, a tarefa fica visível como solicitação humana.

2. Nomes E Compatibilidade

O nome público do produto é **MaestrIA**.

O codinome histórico **wuphf** ainda aparece no binário, no pacote npm, em comandos, imports internos, caminhos e alguns scripts por compatibilidade. Isso permite evoluir a marca sem quebrar automações existentes.

Exemplos:

```
npx wuphf
wuphf init
wuphf doctor
```

Esses comandos sobem e operam a MaestrlA mesmo que o nome tecnico continue `wuphf` .

3. Modelo Local-First

A MaestrlA roda como runtime local. O nucleo nao depende de SaaS proprio:

- broker local na porta padrao `7890` ;
- interface web local na porta padrao `7891` ;
- estado local do escritorio;
- repositorios e worktrees na maquina;
- providers de IA chamados diretamente pelo runtime escolhido;
- integracoes externas apenas quando configuradas.

Quando o app esta ocioso, agentes nao ficam consumindo tokens por polling. Eles sao acordados por eventos.

4. Arquitetura Em Alto Nivel

Fluxo principal:

```
humano -> Web UI/TUI/CLI -> broker local -> launcher -> runner do agente -> worktree -> broker -> UI
```

Componentes principais:

Componente	O que faz
<code>cmd/wuphf/</code>	Entrada da CLI, comandos, TUI, onboarding e launcher local
<code>internal/team/broker.go</code>	Barramento de mensagens, tarefas, filas, eventos e estado do escritorio
<code>internal/team/launcher.go</code>	Decide quais agentes acordam para uma mensagem ou tarefa
<code>internal/team/headless_*.go</code>	Executa providers como turnos headless, novos a cada turno
<code>internal/team/worktree.go</code>	Cria e gerencia worktrees isoladas para agentes
<code>internal/team/resume.go</code>	Retoma tarefas e mensagens pendentes depois de reinicio
<code>internal/teammcp/</code>	Expoe ferramentas MCP escopadas para cada agente e modo
<code>internal/provider/</code>	Providers diretos como Gemini, Codex, Claude Code e Ollama
<code>internal/action/</code>	Action providers opcionais, como Composio e <code>one</code>
<code>web/</code>	Interface web local

Componente	O que faz
templates/	Blueprints, employees e starter kit
mcp/	Servidores MCP auxiliares

5. Quando Cada Coisa Acontece

Ao iniciar

1. A CLI lê configuração, flags e estado local.
2. O broker local sobe.
3. A UI web ou TUI é iniciada.
4. O escritório carrega agentes, canais, tarefas, histórico e integrações configuradas.
5. Se existir trabalho inacabado ou mensagens humanas sem resposta, a rotina de recovery pode ressurgir esse contexto.

Exemplo:

```
npx wuphf
```

Resultado esperado:

```
Broker local em 127.0.0.1:7890
Web UI em http://localhost:7891
Escritório aberto no navegador, a menos que --no-open tenha sido usado.
```

Ao enviar uma mensagem

1. A UI posta a mensagem no broker.
2. O broker salva o evento no canal correto.
3. O launcher verifica menções, modo focus/collab, canal, tarefa e contexto.
4. Agentes elegíveis são acordados.
5. A resposta streama de volta para o broker e aparece na UI.
6. Menções feitas por agentes podem acordar outros agentes.

Exemplo:

```
@eng implemente uma checagem para validar o payload antes de salvar.
```

Quando isso é feito:

- imediatamente após o envio da mensagem;
- apenas para agentes relevantes;
- sem polling ocioso.

Ao criar ou atualizar uma tarefa

Tarefas aparecem quando o humano cria manualmente, quando um agente registra trabalho ou quando um fluxo operacional gera trabalho a partir de blueprint ou request.

Estados comuns:

Estado	Significado	Proximo caminho esperado
open / pending	Trabalho existe, mas ainda nao esta claramente em execucao	atribuir, detalhar ou deixar no backlog
in_progress	Trabalho tem dono ativo	runner ativo, continuacao enfileirada ou dono visivel
review / in_review	Pausado para revisao ou aprovacao	revisor nomeado, request pendente ou dono humano
blocked	Nao pode prosseguir com segurancia	bloqueio explicito e acao necessaria
done	Concluido	nenhum runner deve continuar
canceled	Interrompido intencionalmente	nenhum runner deve continuar

Exemplo via slash command:

```
/task claim task-123
/task block task-123 aguardando token do Telegram
/task done task-123 validado no build local
```

Quando uma tarefa precisa de voce

Se uma tarefa exige decisao humana, comando, credencial, revisao ou aprovacao, ela deve aparecer como request ou item de atencao.

O operador pode responder pela UI, pela aba de solicitacoes ou por canal externo configurado, como Telegram.

Exemplo:

```
Agente: Preciso que voce confirme se posso rodar a migracao.
Voce: aprovado, rode em ambiente local primeiro.
```

Ao concluir uma tarefa

Uma tarefa deve deixar rastro auditavel:

- o que foi tentado;
- o que mudou;
- onde estao arquivos, logs ou artefatos;
- o que foi verificado;
- o que ficou pendente ou bloqueado.

Para código, uma conclusão saudável normalmente inclui arquivos alterados, validação executada e riscos restantes.

6. Como Subir E Operar

Execução rápida

```
npx wuphf
```

Instalação global

```
npm install -g wuphf
wuphf
```

Build local por fonte

```
git clone https://github.com/luiztrilha/dunderia.git
cd dunderia
go build -o wuphf.exe ./cmd/wuphf
.\wuphf.exe
```

Inicialização persistente

```
wuphf init
```

`wuphf init` salva padrões em `~/.wuphf/config.json`. Quando cloud backup estiver configurado e acessível, também reidrata estado local leve, como:

- `company.json`;
- `onboarded.json`;
- `cloud-backup-bootstrap.json`;
- `~/.codex/auth.json`;
- `~/.codex/config.toml`;
- `~/.codex/skills`;
- `~/.agents/skills`;
- credenciais ADC do Google quando existirem no backup.

Repositórios locais pesados ficam fora desse escopo.

7. Flags Principais

Flag	Quando usar	O que faz
<code>--provider <name></code>	Quando quiser escolher runtime de IA	Usa <code>claude-code</code> , <code>codex</code> , <code>gemini</code> ou <code>ollama</code>

Flag	Quando usar	O que faz
<code>--blueprint <id></code>	Ao iniciar a partir de uma operacao pronta	Carrega blueprint operacional
<code>--pack <id></code>	Compatibilidade com presets antigos	Alias legado para blueprint/preset
<code>--from-scratch</code>	Ao criar escritorio novo por diretiva	Ignora configuracao salva e sintetiza operacao
<code>--1o1</code>	Quando quiser conversar com um agente especifico	Abre modo 1:1, padrao <code>ceo</code>
<code>--tui</code>	Quando preferir terminal	Usa TUI em <code>tmux</code>
<code>--no-open</code>	Em servidor ou automacao	Nao abre navegador automaticamente
<code>--broker-port <n></code>	Quando a porta 7890 estiver ocupada	Altera porta do broker
<code>--web-port <n></code>	Quando a porta 7891 estiver ocupada	Altera porta da web UI
<code>--threads-collapsed</code>	Quando quiser UI mais compacta	Inicia threads recolhidas
<code>--memory-backend none</code>	Modo suportado hoje	Desativa memoria organizacional compartilhada externa
<code>--opus-ceo</code>	Quando quiser CEO mais forte	Troca CEO para Opus quando provider suportar
<code>--collab</code>	Quando quiser todos mais ativos	Inicia em modo colaborativo
<code>--unsafe</code>	Apenas desenvolvimento controlado	Ignora algumas checagens de permissao
<code>--cmd <cmd></code>	Automacao nao interativa	Executa comando sem abrir fluxo interativo

Ajuda completa:

```
wuphf --help-all
```

8. Comandos Da CLI

Comando	O que faz	Quando usar
<code>wuphf init</code>	Configura padroes e restaura estado leve quando disponivel	Primeiro uso ou troca de maquina
<code>wuphf shred</code>	Encerra sessao em execucao preservando estado	Parar runtime legado sem destruir escritorio
<code>wuphf import --from legacy</code>	Importa estado de orquestrador externo ou arquivo	Migracao
<code>wuphf log</code>	Lista recibos recentes de tarefas	Auditoria

Comando	O que faz	Quando usar
<code>wuphf log <taskID></code>	Mostra log de tarefa especifica	Investigar uma entrega
<code>wuphf repair-channel-memory</code>	Reconstrui memoria de canais pelo historico do broker	Corrigir historico/memoria local
<code>wuphf doctor</code>	Inspeciona setup e runtime vivo	Diagnostico
<code>wuphf doctor --smoke</code>	Roda checagens basicas de fumaca	Validacao local
<code>wuphf doctor --json</code>	Emite diagnostico em JSON	Automacao

`wuphf shred` preserva canais, agentes, mensagens, tarefas, company e workflows. A destruicao completa foi removida da CLI e da web UI.

9. Interface Web

A interface web padrao fica em:

```
http://localhost:7891
```

Ela e o escritorio principal da MaestrIA.

Areas principais:

Area	O que faz
Home	Visao inicial do escritorio e estado geral
Canais	Conversas compartilhadas do time
DMs	Conversas diretas com agentes
Activity	Movimento recente e atencao operacional
Inbox / Tasks	Tarefas, estados, donos, bloqueios e acoes
Deliveries	Entregas e artefatos agrupados para revisao
Requests	Solicitacoes humanas e aprovacoes
Skills	Habilidades disponiveis e invocacoes
Browser Lab	Superficie para testes/experimentos de navegador quando disponivel
Health / Doctor	Diagnostico de runtime e readiness
Settings / Config	Providers, integracoes, secrets e preferencias
Studio	Pacotes, bootstrap, workflows, dev console e contexto ativo

Canais

Canais organizam conversas publicas do escritorio. Cada canal tem slug, nome, membros e historico.

Quando um canal publico e criado pela UI, o broker grava o canal no `company.json` alem do estado operacional. Isso faz o canal continuar existindo depois de rebuild ou restart, ja que o startup reconcilia a topologia ativa a partir do manifesto.

Exemplo:

```
Canal: general
Mensagem: @ceo priorize as tres tarefas mais importantes para hoje.
```

Quando usado:

- para alinhamento entre agentes;
- para trabalho publico e auditavel;
- para coordenar tarefas que envolvem mais de um papel.

DMs e 1:1

DMs servem para conversar diretamente com um agente.

Exemplo na UI:

```
/1o1 ceo
```

Exemplo pela inicializacao:

```
wuphf --1o1 ceo
```

Quando usado:

- perguntas diretas;
- contexto sensivel;
- alinhar uma tarefa antes de levar ao canal.

Threads

Threads agrupam respostas relacionadas a uma mensagem ou tarefa.

Mensagens novas no topo do canal sao tratadas como pedidos independentes no pacote enviado aos agentes. O launcher nao mistura automaticamente o historico recente nem a memoria do canal nesses turnos; contexto adicional entra quando a mensagem ja pertence a uma thread ou quando vem de uma tarefa.

Ao excluir uma mensagem, o broker remove tambem a copia sintetica guardada na memoria do canal. Isso evita que mensagens apagadas reaparecam na timeline por terem sido restauradas de `shared_memory`.

Quando usado:

- quando uma conversa precisa de detalhe sem poluir o canal;
- quando uma decisao pertence a um pedido especifico;
- quando uma tarefa recebe revisoes e feedbacks.

Inbox / Tasks

A inbox mostra o que esta aberto, em andamento, bloqueado, em review ou concluido. Ela tambem permite acoes como claim, release, pause, resume, feedback, artifact, plan revision e skill proposal quando suportado pela tarefa.

Artefatos de tarefa aceitam `browser_inspection` como evidencia estruturada para trabalho de frontend. Esse tipo guarda URL da pagina, seletor/elemento observado, texto visto, viewport e caminho de screenshot, e entra no mesmo contrato de evidencia, activity ledger, execution trace e resume pack dos demais work products.

Para empacotar essas evidencias para o proximo turno de frontend, use:

```
GET /browser/inspection-handoff-preview?channel=general&viewer_slug=human
GET /browser/inspection-handoff-preview?all_channels=true&viewer_slug=human&task_id=task-123
```

O retorno e read-only e inclui `persisted:false`, `ready`, campos ausentes, sinais de risco, URL, seletor, texto observado, viewport, screenshot, resumo de evidencia e um `handoff_prompt`. Ele nao abre browser, nao cria artefato e nao altera tarefa; apenas organiza o que ja foi registrado como `browser_inspection`.

Exemplo:

```
Tarefa: Implementar alerta no Telegram
Owner: eng
Status: review
Next: humano validar no Telegram real
```

Requests

Requests sao pedidos de decisao humana ou aprovacao.

Exemplo:

```
Pedido: Posso enviar email via Composio?
Resposta: Sim, envie apenas para o contato aprovado e anexe o resumo.
```

Quando usado:

- ferramentas mutantes;
- comandos que exigem permissao;
- informacao que o agente nao tem;
- decisao de produto;
- revisao humana.

Deliveries

Deliveries agrupam entregas e artefatos para revisao. Uma delivery pode apontar para tarefas, arquivos, logs ou resultados que exigem atencao.

Tasks e requests arquivados nao aparecem na visao normal de deliveries. Para auditoria historica, use `include_done=true`; itens arquivados so aparecem quando a chamada tambem pede `include_archived=true`.

Quando usado:

- revisar trabalho final;
- comparar entregas por repositório;
- ver o que aguarda você.

Scheduler Skill Preview

Scheduler skill preview mostra, sem alterar jobs, se as rotinas agendadas estão ligadas a skills existentes e confiáveis.

Endpoint:

```
GET /scheduler/skill-preview?channel=general&viewer_slug=human
```

O retorno inclui:

- `persisted:false` ;
- `skill_names` vinculadas ao job, incluindo compatibilidade com `skill_name` ;
- readiness por job: `ready` , `warning` ou `blocked` ;
- risco por skill, score de confiança, provenance, scan status e capabilities;
- sinais como skill ausente, skill proposta, baixa confiança ou referência a mutação do próprio scheduler.

Quando usado:

- revisar rotinas antes de confiar nelas;
- encontrar jobs agendados com skills ausentes ou arriscadas;
- preparar jobs com múltiplas skills sem acionar execução automática;
- manter automações agendadas auditáveis antes de ampliar permissão.

Scheduler Revisions Preview

Scheduler revisions preview mostra, sem alterar jobs, quais rotinas ainda não têm histórico append-only, save com conflito, bloqueio de edição suja, confirmação de restore e auditoria de rollback.

Endpoint:

```
GET /scheduler/revisions-preview?channel=general&viewer_slug=human
```

O retorno inclui `persisted:false` , jobs visíveis, `revision_count` , `restore_enabled:false` , `restore_readiness` , políticas ausentes, ações bloqueadas e `next_step` . O endpoint não grava revisões, não restaura rotina e não substitui job agendado.

Toolset Profile Preview

Toolset profile preview mostra, sem alterar agentes ou canais, quais ferramentas e capacidades estão efetivamente disponíveis por agente/canal.

Endpoint:

```
GET /toolsets/profile-preview?channel=general&viewer_slug=human
```

O retorno inclui:

- `persisted:false` ;
- agente, canal, permission mode e `allowed_tools` declarados;
- toolsets inferidos, como office, memory, skills, tasks, requests, scoped-mcp, nex ou topology;
- capabilities vindas de MCP local, adapters e skills;
- marcadores de risco: mutating, external, secret-bearing e scheduler-mutating;
- drift entre ferramentas declaradas e runtime real;
- acao sugerida: `keep` , `document` , `review` ou `restrict` .

Quando usado:

- auditar permissao efetiva antes de ampliar uma automacao;
- encontrar ferramentas disponiveis no runtime mas nao documentadas no agente;
- comparar risco por canal/agente sem mexer na topologia;
- preparar uma futura politica de toolsets por perfil.

Human Permissions Preview

Human permissions preview mostra, sem criar usuarios ou alterar canais, quais capacidades um viewer teria por canal.

Endpoint:

```
GET /humans/permissions-preview?channel=general&viewer_slug=human
GET /humans/permissions-preview?all_channels=true&viewer_slug=human
```

O retorno inclui:

- `persisted:false` ;
- snapshots por canal com `access_level` , `can_read` , `can_answer_requests` , `can_review_tasks` , `can_approve_actions` e `can_mutate_topology` ;
- capacidades por snapshot, como `channel.read` , `request.answer` , `task.review` , `action.approve` e `topology.mutate` ;
- `topology.mutate` sempre bloqueado nessa superficie;
- `signals` e `next_step` para distinguir viewer somente leitura, operador aprovador e canal protegido.

Quando usado:

- revisar limites antes de compartilhar uma superficie com outro operador;
- separar leitura, resposta/aprovacao e mutacao de topologia;
- manter multi-human como modelo de permissao auditavel, sem criar sharing casual;
- preservar a regra de topologia protegida para agentes, canais, blueprints e estado.

Multi-Company Control Plane Preview

Multi-company control plane preview mostra como um futuro controle de varias empresas teria que isolar estado, exportar pacotes e bloquear mutacoes de topologia. Ele nao cria empresas, nao troca `company.json` , nao importa pacote, nao reponta `broker-state.json` e nao aplica alteracoes em agentes ou canais.

Endpoint:

```
GET /companies/control-plane-preview
```

O retorno inclui:

- `persisted:false` ;
- `current_company` com nome, lead, fonte do manifesto e contagens atuais de membros, canais, tarefas, skills e adapters;
- `export_items` para manifesto, membros, canais, tarefas, skills e adapters, todos `preview_only:true` e `secret_scrubbed:true` ;
- `isolation` com contratos exigidos para state roots, topologia protegida e secret scrub;
- `blocked_mutations` como `create_company` , `switch_company` , `import_company` , `delete_company` , `topology_apply` e `state_root_repoint` ;
- `required_policies` e `missing_policies` para identidade de empresa, raiz de estado, scrub de secrets, backup, colisao, rollback e revisao humana;
- `apply_enabled:false` e `topology_mutation_enabled:false` .

Quando usado:

- revisar o desenho de multi-company sem criar uma segunda empresa;
- preparar export/import apenas como pacote sanitizado e revisavel;
- detectar quais politicas faltam antes de qualquer isolamento real;
- manter a regra de topologia protegida mesmo em fluxos de portabilidade.

Recall Search Preview

Recall search preview procura, sem gravar nada, contexto reutilizavel em mensagens, tarefas, artefatos, decisoes e conhecimento promovido.

Endpoint:

```
GET /recall/search-preview?channel=general&viewer_slug=human&q=release
```

O retorno inclui:

- `persisted:false` ;
- resultados ranqueados por `kind` , como `message` , `task` , `artifact` , `decision` ou `knowledge` ;
- resumo curto em vez de transcript bruto;
- referencias de origem para voltar ao item fonte;
- sinais de ranking, como `match` em titulo, resumo, corpo ou vinculo de tarefa;
- `quality_score` e `quality_signals` , calculados por evidencia, backlinks de tarefa, multiplas fontes, frescor, resumo e penalidades de risco;
- sinais de risco, como conteudo parecido com segredo, fonte longa resumida ou referencia externa;
- filtros por `kind` , `channel` , `viewer_slug` , `limit` e `all_channels` .

Quando usado:

- perguntar "onde vimos isso antes?" sem escavar historico manualmente;
- reaproveitar decisoes, evidencias e artefatos passados;
- alimentar revisao humana antes de transformar recall em memoria ou skill;
- manter recall auditavel e sem inserir transcript bruto automaticamente no prompt.

Command Manifest

Command manifest lista comandos slash e a semantica operacional de cada um sem executar nada.

Endpoint:

```
GET /commands/manifest?surface=web
GET /commands/manifest?surface=tui
GET /commands/manifest?surface=all
GET /commands/manifest-drift?surface=web
```

O retorno inclui:

- `persisted:false` ;
- nome do comando, categoria e descricao;
- rota relacionada quando existe;
- se o comando e mutante;
- se exige confirmacao;
- se toca uma area sensivel de topologia;
- sinais como `runtime_reset` , `task_mutation` , `protected_topology` ou `agent_signal` .

O filtro `surface` aceita `web` , `tui` ou `all` . Quando omitido, o broker retorna `web` para manter o Studio web restrito aos comandos que ele sabe executar.

Quando usado:

- manter help, autocomplete, manual e UI alinhados;
- revisar comandos mutantes antes de expor novas superficies;
- gerar menus ou paletas a partir de uma fonte comum;
- evitar drift entre documentacao e comportamento.

No Studio web, o autocomplete do compositor, os resultados de comandos no Cmd+K e o texto de `/help` usam `/commands/manifest` quando o broker esta acessivel. A UI conserva uma lista local de fallback para continuar operando durante reconnect ou antes do manifest carregar.

A TUI tambem deriva o autocomplete slash do mesmo manifest usando `surface=tui` , incluindo comandos de terminal como `/messages` , `/request` , `/channel` , `/agent prompt` , `/reset-dm` e `/quit` .

`/commands/manifest-drift` compara o manifest filtrado pela superficie com a secao de slash commands do manual e retorna `persisted:false` , `status` , `summary` e itens de drift. Sem `surface` , a comparacao continua web-only. Os tipos principais sao `manifest_missing_manual` e `manual_missing_manifest` .

Execution Environments Preview

Execution environments preview mostra, sem iniciar backends, quais ambientes de execucao existem ou estao planejados.

Endpoint:

```
GET /runtime/execution-environments-preview?channel=general&viewer_slug=human
```

O retorno inclui:

- `persisted:false` ;
- ambientes como `office` , `local_worktree` , `external_workspace` e `live_external` ;
- contagem de workspaces, tarefas, canais e sinais;

- readiness por ambiente;
- adapters futuros como `docker` e `ssh` marcados como `blocked` ate existir politica governada;
- para Docker/SSH, `required_policies`, `missing_policies`, `policy_checks` e `next_step`;
- checks read-only de binario no `PATH`, sem executar Docker, abrir SSH, conectar em host remoto ou iniciar backend.

Quando usado:

- revisar onde o trabalho pode rodar antes de ampliar automacao;
- detectar workspace degradado ou sujo;
- preparar suporte futuro a Docker/SSH sem habilita-los por padrao;
- ver quais politicas faltam para workspace, secrets, rede, cleanup, auditoria, aprovacao, host e chave;
- explicar risco de execucao fora do broker local.

Remote Sandbox Preview

Remote sandbox preview aprofunda a parte remota de execution environments sem habilitar execucao fora do broker local. Ele lista candidatos Docker, SSH e self-hosted worker, mas mantem todos `execution_enabled:false`.

Endpoint:

```
GET /runtime/remote-sandbox-preview
```

O retorno inclui:

- `persisted:false`;
- candidatos `docker`, `ssh` e `self_hosted_worker`;
- `readiness` por candidato, normalmente `blocked` ate existirem politicas governadas;
- `install_command_policy`, `install_command_enabled:false` e `install_command_preview` para comandos de instalacao por adapter;
- `required_policies`, `missing_policies`, `policy_checks`, `risk_signals` e `next_step`;
- checks de binario para Docker/SSH quando aplicavel, sem abrir container, SSH, worker remoto, comando de instalacao, rede ou workspace externo.

Quando usado:

- transformar "sandbox remoto" em checklist de governanca antes de produto;
- separar disponibilidade de binario de autorizacao para executar;
- revisar politicas de workspace, secrets, rede, cleanup, auditoria, aprovacao, host, chave e custo;
- manter qualquer fleet remota como desenho bloqueado ate existir apply path explicito.

Plugin Sandbox Preview

Plugin sandbox preview mostra quais adapters, skills/plugin-like items e workers health-only poderiam entrar em um futuro sandbox e quais politicas ainda faltam. O unico worker embutido hoje e `worker:noop-health`: ele declara manifesto, health check estatico, filesystem `none`, rede `none` e sem secrets, mas nao executa acoes de plugin, shell, chamadas de rede ou escrita em disco.

Endpoint:

```
GET /plugins/sandbox-preview
```

O retorno inclui:

- `persisted:false` ;
- candidatos derivados de adapters, skills ativos e workers health-only embutidos;
- status de sandbox por item, como `ready` , `review` ou `blocked` ;
- capacidades declaradas e politicas exigidas;
- politicas ausentes como `manifest` , `capabilities` , `health_check` , `filesystem_scope` , `network_policy` e `secret_refs` ;
- metadados de worker quando existirem, como `worker_class` , `manifest_id` , `manifest_signature` , `health_check` , `filesystem_scope` , `network_policy` e `secret_refs` ;
- referencias de configuracao redigidas quando necessario;
- sinais de risco e proximo passo sugerido.

Quando usado:

- revisar risco antes de permitir workers fora do processo;
- impedir que plugin com filesystem, rede ou segredo sem escopo vire execucao automatica;
- preparar manifestos e health checks sem rodar processo local arbitrario;
- validar o caminho inicial de sandbox com um worker no-op que so reporta saude;
- manter Paperclip-style plugin execution bloqueado ate existir sandbox real para workers que executam acoes.

Marketplace Manifest Preview

Marketplace manifest preview define o contrato inicial de um futuro marketplace de skills/plugins sem instalar ou atualizar nada. Ele reaproveita sinais de trust, provenance, capability-upgrade, adapters e sandbox para mostrar assinatura, hash e drift antes de qualquer apply path existir.

Endpoint:

```
GET /marketplace/manifest-preview
```

O retorno inclui:

- `persisted:false` ;
- `status` geral e `summary` por estado;
- manifestos derivados de skills ativos, adapters e do worker embutido `worker:noop-health` ;
- `manifest_id` , `manifest_version` , `manifest_signature` e `signature_status` ;
- `installed_hash` , `expected_hash` e `drift_status` ;
- `trust_level` , `trust_score` , capacidades atuais/propostas e `added_capabilities` ;
- `required_reviews` , `required_policies` , `missing_policies` , `risk_signals` , `risk_score` e `risk_level` ;
- `install_enabled:false` e `update_enabled:false` para deixar claro que a superficie e design-only.

Quando usado:

- revisar supply-chain risk antes de criar um instalador;
- detectar hash/provenance/capability drift em skills locais;
- checar adapters com config refs suspeitas sem expor segredos;
- usar o no-op worker como exemplo de manifesto health-only;
- manter install/update de marketplace bloqueado ate existir assinatura confiavel, scan de conteudo baixado, revisao de capacidades, rollback e confirmacao explicita.

Skills

Skills são capacidades reutilizáveis que podem orientar agentes, executar fluxos ou expor metadados.

Quando usado:

- repetir um procedimento com segurança;
- encapsular um workflow local;
- ensinar agentes a seguir regras específicas.

Skills também carregam provenance opcional:

- `source_type` e `source_ref` indicam origem, como starter pack, proposta local ou aprendizado promovido de tarefa;
- `source_hash` identifica a versão do conteúdo;
- `installed_at`, `last_scanned_at`, `scan_status` e `scan_summary` mostram instalação e checagem estática local;
- `/skills/trust` inclui esses sinais no registro de confiança;
- `/skills/provenance-preview` sugere, em dry-run, quais skills antigas precisam receber origem, hash ou scan.

Skill file preview expõe arquivos virtuais de uma skill por disclosure progressivo:

```
GET /skills/release-playbook/files-preview
GET /skills/release-playbook/files-preview?file=content.md
```

Sem `file=`, o endpoint lista arquivos como `metadata.json`, `content.md`, `workflow.json`, `trigger.txt` e `scan.txt` sem devolver conteúdo completo. Com `file=`, devolve apenas o arquivo solicitado, com sinais de risco quando o conteúdo parece conter segredo.

Learning Candidates

Learning candidates são previews read-only de tarefas concluídas que parecem boas fontes para aprendizado reutilizável. Eles ajudam o operador a ver quais experiências podem virar skills sem permitir que o agente crie ou altere habilidades automaticamente.

Endpoint:

```
GET /learning/candidates?channel=general&viewer_slug=human&q=release
GET /learning/candidates/diff-preview?task_id=task-123&channel=general&viewer_slug=human
```

O retorno inclui:

- tarefa de origem;
- skill sugerida, como `learned-task-123`;
- sinais que justificam o candidato, como evidência de resultado, artefatos, plano, feedback ou findings;
- provenance com trechos de evidência, plano, artefatos ou feedback;
- indicador de que a skill já foi promovida quando existir.

O diff preview de um candidato é também read-only. Ele monta a mesma skill que seria criada por `promote_learning`, mas retorna `persisted: false`, `action`, `duplicate`, `proposed_skill`, `existing_skill` quando houver, `risk_level`, `risk_signals` e arquivos virtuais como `metadata.json`, `content.md` e `provenance.json`. Com `include_content=true`, o operador vê o conteúdo que seria escrito antes de promover.

A tela de detalhe da tarefa mostra o botão `Prev` no bloco de aprendizado organizacional. Essa `prev` abre o diff virtual, mostra se a ação criaria uma skill ou se já existe duplicata, exibe risco local e deixa claro que nada foi persistido.

Quando usado:

- revisar aprendizados antes de promover uma skill;
- encontrar workflows repetíveis depois de uma entrega;
- auditar de onde veio uma sugestão de aprendizado;
- manter o loop de aprendizado humano-aprovado e sem mutação automática.

Memory Curation Preview

Memory curation preview mostra, sem gravar nada, quais mensagens, tarefas e decisões parecem boas candidatas para memória de canal.

Endpoint:

```
GET /memory/curation-preview?channel=general&viewer_slug=human&q=release
```

O retorno inclui:

- `persisted:false` ;
- origem do candidato, como `message` , `task` ou `decision` ;
- namespace e chave de memória sugeridos;
- ação sugerida: `remember` , `consolidate` , `review` ou `discard` ;
- score, confidence, sinais de utilidade e sinais de risco;
- `include_discard=true` para ver também candidatos rejeitados por baixo sinal ou conteúdo sensível.

Quando usado:

- revisar o que merece entrar na memória antes de persistir;
- procurar memória potencial por canal ou termo;
- identificar conteúdo com cara de segredo antes que vire contexto reutilizável;
- consolidar memória existente sem mutação automática.

Knowledge Wiki Preview

Knowledge wiki preview projeta entradas existentes de conhecimento em artigos markdown citados, sem gravar wiki, memória ou arquivo.

Endpoint:

```
GET /knowledge/wiki-preview?channel=general&viewer_slug=human&q=release
GET /knowledge/wiki-lint?channel=general&viewer_slug=human
GET /knowledge/wiki-promotion-preview?channel=general&viewer_slug=human&task_id=task-123
```

O retorno inclui `persisted: false` , artigos com `slug` , `title` , `kind` , `channel` , `summary` , `markdown` , `sources` , `backlinks` , `stale` e `risk_signals` . A fonte de cada artigo aponta para a entrada de conhecimento existente, como tarefa concluída ou skill de aprendizado.

`/knowledge/wiki-lint` reaproveita essa projecao e retorna findings read-only com `severity` , `kind` , `summary` , artigo, canal e fonte relacionada. Os checks atuais cobrem fonte ausente ou incompleta, fonte antiga, conteudo com cara de segredo e backlink de tarefa quebrado.

`/knowledge/wiki-promotion-preview` monta a etapa seguinte sem persistir nada. Ele retorna propostas com `target_path` , `markdown` , `diff` , `commit_message` , `required_reviews` , `lint_findings` , `reviewed_commit_only:true` e sinais como `shared_memory_not_mutated` . A proposta e sempre um diff revisavel para commit git normal; ela nao cria arquivo, nao escreve memoria compartilhada e nao faz commit automatico.

`/knowledge/wiki-editor-preview` mostra a prontidao de um futuro editor source/rich sem habilitar edicao. Ele lista modos com `editor_enabled:false` e checks de round-trip markdown, preservacao de wikilinks, seguranca de blocos de codigo, restore de rascunho, conflito e acessibilidade.

Quando usado:

- revisar como uma wiki futura ficaria antes de criar arquivos;
- encontrar entradas antigas ou arriscadas por `stale` e `risk_signals` ;
- preparar uma etapa de review humano antes de qualquer escrita git-native;
- transformar evidencia de tarefa ou aprendizado em patch markdown revisavel;
- bloquear promocao quando lint detectar segredo, fonte quebrada ou outro risco.

Health / Doctor

Doctor mostra readiness, runtime, build web, processos, broker e smoke checks.

Exemplo:

```
/doctor
```

ou:

```
wuphf doctor --smoke
```

Quando usado:

- broker parece offline;
- UI nao conecta;
- provider nao responde;
- worktree ou memoria parece inconsistente;
- antes de diagnosticar bugs de runtime.

10. Slash Commands Na Web

Comando	O que faz
<code>/clear</code>	Limpa mensagens do canal atual no broker e recarrega o feed visivel
<code>/help</code>	Mostra comandos disponiveis
<code>/requests</code>	Abre solicitacoes

Comando	O que faz
<code>/policies</code>	Abre politicas ativas, incluindo regras restauradas de backup
<code>/skills</code>	Abre habilidades
<code>/calendar</code>	Abre calendario quando disponivel
<code>/tasks</code>	Abre inbox de tarefas
<code>/recover</code>	Abre health check/recovery
<code>/doctor</code>	Abre health check/recovery
<code>/threads</code>	Abre visao de threads
<code>/provider</code>	Abre seletor de provider
<code>/search</code>	Abre busca
<code>/focus</code>	Ativa modo foco
<code>/collab</code>	Desativa modo foco e usa modo colaborativo
<code>/pause</code>	Pausa agentes
<code>/resume</code>	Retoma agentes
<code>/reset</code>	Reinicia mensagens, tarefas e solicitacoes de runtime, preservando topologia, membros e politicas
<code>/lo1 <agent></code>	Abre DM com um agente
<code>/task <action> <id></code> <code>[detalhes]</code>	Aplica acao em uma tarefa
<code>/cancel <id></code>	Libera/cancela acompanhamento de tarefa

Exemplos:

```

/focus
/lo1 designer
/task block task-42 aguardando confirmacao do cliente
/requests

```

11. Modos De Trabalho

Focus mode

No modo foco, o broker acorda menos agentes. A regra geral e manter o CEO ou agente diretamente mencionado como ponto de entrada.

Quando usar:

- reduzir ruido;
- evitar que muitos agentes respondam;
- conduzir uma decisao centralizada.

Exemplo:

```
/focus
@ceo organize as prioridades e so chame engenharia quando necessario.
```

Collab mode

No modo colaborativo, mais agentes podem responder conforme o contexto.

Quando usar:

- brainstorming operacional;
- tarefas multidisciplinares;
- planejamento com varios papeis.

Exemplo:

```
/collab
Precisamos lancar uma pagina nova, alinhem produto, design e engenharia.
```

1:1

O modo 1:1 conversa com um agente especifico e reduz o escopo de ferramentas carregadas.

Quando usar:

- investigacao direcionada;
- conversa privada com um papel;
- respostas rapidas de um especialista.

12. Providers De Runtime

A MaestrIA pode executar agentes usando providers diferentes.

Provider	Como selecionar	Observacao
Claude Code	padrao ou <code>--provider claude-code</code>	Usa CLI local do Claude Code
Codex	<code>--provider codex</code>	Usa runtime Codex local
Gemini	<code>--provider gemini</code>	Usa API Gemini
Ollama	<code>--provider ollama</code>	Usa modelo local ja baixado

Exemplo:

```
wuphf --provider codex
wuphf --provider ollama
```

Como e feito:

- cada turno cria uma execucao nova do provider;
- antes da execucao, o runtime resolve automaticamente um perfil de modelo dentro da mesma familia do provider ativo;
- o prompt recebe contexto do canal, tarefa, agente e ferramentas escopadas;
- a saida e transmitida ao broker;
- a worktree do agente isola arquivos e comandos.

Para Ollama, o cliente HTTP nativo segue o timeout do turno do launcher em vez de impor um limite proprio menor. Isso evita cancelar modelos locais lentos antes do prazo operacional do runner.

Roteamento automatico de modelo

O roteamento de modelo e conservador e family-native: Claude escolhe apenas modelos Claude, Codex escolhe apenas modelos Codex/OpenAI, Gemini escolhe apenas modelos Gemini, e Ollama permanece no modelo local configurado.

Ordem de decisao:

1. runtime_model explicito da tarefa vence quando pertence a familia do provider.
2. Modelo fixado no provider do agente vence quando pertence a familia do provider.
3. O runtime escolhe um perfil automatico: fast, balanced, deep ou premium.

Perfis mais fortes so aparecem com sinais concretos, como execucao em workspace, reasoning_effort alto, contexto grande, ambiguidade estrategica, debug, regressao, refatoracao, migracao, seguranca ou pedido humano explicito por modelo mais forte.

Visibilidade:

- cada turno registra model-routing no log headless do agente;
- as linhas de latencia incluem provider, perfil e modelo;
- o detalhe de progresso mostra o perfil/modelo enquanto o agente pensa;
- nao ha hard-stop de orcamento automatico nessa politica.

Overrides por ambiente:

Use WUPHF_MODEL_ROUTE_<PROVIDER>_<PROFILE> , com provider em maiusculas e hifen trocado por sublinhado. Exemplos:

Variavel	Uso
WUPHF_MODEL_ROUTE_CLAUDE_CODE_DEEP	Modelo Claude para perfil deep
WUPHF_MODEL_ROUTE_CODEX_PREMIUM	Modelo Codex/OpenAI para perfil premium
WUPHF_MODEL_ROUTE_GEMINI_FAST	Modelo Gemini API para perfil fast
WUPHF_MODEL_ROUTE_OLLAMA_BALANCED	Modelo local Ollama para perfil balanced

Quando e feito:

- ao acordar agente por mensagem;
- ao retomar tarefa;

- ao processar request aprovado;
- ao executar continuacao enfileirada.

13. Runners Novos Por Turno

A MaestrIA evita sessoes persistentes gigantes. Cada turno do agente e uma execucao headless nova.

Vantagens:

- contexto mais controlado;
- menos acumulacao invisivel;
- melhor cache de prompt quando o provider suporta;
- falhas ficam isoladas por turno;
- recovery depende do estado do broker, nao de uma sessao opaca.

Exemplo conceitual:

Mensagem humana -> monta prompt do agente -> executa provider uma vez -> salva resposta -> encerra runner .

14. MCP Escopado Por Agente

Ferramentas MCP sao carregadas conforme agente e modo.

Como e feito:

- DM carrega conjunto menor de ferramentas;
- office mode carrega mais ferramentas;
- agentes recebem ferramentas coerentes com seu papel;
- ferramentas mutantes passam por request/aprovacao quando necessario.
- skills usam disclosure progressivo: `team_skill_list` lista metadados compactos, `team_skill_view` abre uma skill especifica sem registrar invocacao, e `team_skill_run` continua sendo o caminho auditado que incrementa uso e registra `skill_invocation` .

Quando importa:

- reduz custo de schema;
- evita expor ferramentas desnecessarias;
- melhora previsibilidade;
- limita superficie de risco.

15. Worktrees Isoladas

Cada agente pode trabalhar em uma worktree Git isolada.

Como e feito:

- o runtime cria ou reaproveita worktree associada ao agente/tarefa;
- comandos do agente rodam naquele diretorio;

- leases e auditorias ajudam a manter a associacao correta;
- tarefas registram path/branch quando aplicavel.

Quando usar:

- desenvolvimento paralelo;
- evitar que agentes pisem no mesmo checkout;
- auditar qual agente mexeu em qual area.
- `execution_mode=external_workspace` usa o diretorio externo informado como `working_directory` ; nesse modo o broker nao cria worktree Git e aceita diretorio existente mesmo sem `.git` , inclusive quando o caminho passa por diretorios ocultos como `.wuphf/cache/...` .

Exemplo:

Tarefa `task-17` -> owner eng -> worktree dedicada -> alteracoes verificadas -> delivery para review.

16. Tarefas, Bloqueios E Review

Tarefas representam trabalho operacional. Elas devem ter proximo passo claro.

Campos e conceitos comuns:

- `id` : identificador da tarefa;
- `title` : titulo humano;
- `status` : estado operacional;
- `owner` : agente ou humano responsavel;
- `channel` : canal de origem;
- `blocker` : impedimento explicito;
- `review_state` : estado de revisao;
- `artifact` : evidencia ou entrega;
- `execution_lock` : trava para evitar execucao duplicada;
- `plan_revision` : revisao do plano;
- `feedback` : comentario de revisao.

Exemplo de tarefa saudavel:

ID: `task-telegram-alerts`
 Titulo: Apagar alerta do Telegram depois de resolvido
 Owner: eng
 Status: done
 Evidencia: testes Telegram passaram; mensagem deletada via `deleteMessage` quando request deixa de estar ativo

Exemplo de tarefa bloqueada:

ID: `task-production-send`
 Status: blocked

Blocker: aguardando aprovacao humana para enviar email real
Owner esperado: human

17. Recovery E Retomada

Recovery e conservador. A Maestria pode ressurgir trabalho inacabado, mas nao deve inventar mudancas perigosas.

Pode acontecer automaticamente:

- ressurgir tarefa apos reinicio quando dono esta claro;
- reencaminhar recibos de tarefa inacabada ao agente certo;
- registrar watchdog/recovery quando runner falha ou silencia;
- manter o mesmo dono ao tentar continuidade.

Nao deve acontecer automaticamente:

- reatribuir trabalho para outro agente sem regra explicita;
- marcar trabalho como done apenas por prosa;
- mudar agentes, canais, company, blueprints ou workflows sem aprovacao atual;
- esconder falhas atras de retries infinitos.

Exemplo:

O app fecha durante uma tarefa em andamento.
Na proxima inicializacao, o recovery detecta a tarefa e cria um caminho visivel:
"task-123 ainda estava in_progress para eng; retomar ou marcar bloqueada?"

18. Memoria E Logs

A memoria organizacional compartilhada externa esta desabilitada no modelo atual.

Modo suportado:

```
wuphf --memory-backend none
```

Ainda persistem:

- historico de canais;
- recibos de tarefa;
- historico salvo do broker;
- mensagens humanas sem resposta;
- notas privadas por agente;
- contexto de rascunho;
- artefatos e logs associados a tarefas.

Comandos uteis:


```
wuphf log
wuphf log task-123
wuphf repair-channel-memory
```

Quando usar:

- auditar o que um agente fez;
- entender por que uma tarefa travou;
- reconstruir memoria de canais;
- preparar handoff depois de reinicio.

19. Blueprints, Packs E Templates

Blueprints descrevem operacoes e times iniciais.

Caminhos principais:

```
templates/operations
templates/employees
templates/starter-kit
```

Presets legados ainda existem:

- `starter` ;
- `founding-team` ;
- `coding-team` ;
- `lead-gen-agency` ;
- `revops` .

Exemplos:

```
wuphf --blueprint minha-operacao
wuphf --pack coding-team
wuphf --from-scratch
```

Quando usar:

- montar escritorio com uma topologia pronta;
- testar um time especifico;
- iniciar operacao nova a partir de diretiva;
- distribuir um starter kit sanitizado.

Regra importante: agentes, canais, workflows salvos e company state sao topologia protegida. Nao altere roster ou canais sem autorizacao explicita do usuario na conversa atual.

No startup, `company.json` e a fonte autoritativa para roster e canais. O broker ainda usa `broker-state.json` para historico operacional, tarefas e mensagens, mas poda agentes, canais e DMs que nao existem mais no manifesto atual para evitar mistura de topologias antigas.

20. Onboarding

O onboarding coleta informacoes iniciais, como empresa, objetivo, provider e chaves quando necessario.

Como e feito:

- via TUI/CLI ou web, conforme fluxo iniciado;
- salva padroes locais;
- pode criar ou selecionar blueprint;
- pode preparar credenciais locais sem enviar secrets ao repo.

Quando acontece:

- primeira execucao;
- quando `wuphf init` e usado;
- quando o estado local indica que o escritorio ainda nao foi configurado;
- quando o operador escolhe recriar a operacao.

21. Integracao Telegram

A integracao com Telegram usa Bot API, nao WhatsApp Cloud API.

O que faz:

- conecta um bot do Telegram ao escritorio;
- mapeia chats/grupos para canais;
- envia mensagens do Telegram para o broker;
- envia mensagens do broker para o Telegram;
- publica alertas de atencao humana;
- apaga automaticamente alertas de atencao humana quando a tarefa/request deixa de estar ativa, quando o bot tem permissao para deletar.

Como conectar:

1. Crie um bot no `@BotFather`.
2. Salve o token no fluxo de configuracao ou secret store.
3. Adicione o bot ao grupo ou abra conversa com ele.
4. Use `/connect` no escritorio.
5. Escolha Telegram.
6. Descubra grupos/chats e mapeie para um canal.

Exemplo:

```
/connect
Provider: Telegram
Chat: Equipe Dev
Canal: telegram-dev
```

Como funciona por baixo:

- `TelegramTransport` long-polla `getUpdates`;

- mensagens recebidas chamam `PostInboundSurfaceMessage` ;
- mensagens do broker entram na fila externa `telegram` ;
- saída usa `sendMessage` ou `sendMessage` com HTML;
- status de digitacao usa `sendChatAction` ;
- alertas humanos sao deduplicados por event ID;
- delecao usa `deleteMessage` para alertas resolvidos.

Quando alertas sao enviados:

- existe tarefa/request aberta que exige acao humana;
- ha canal Telegram mapeado;
- o transporte esta ativo e token disponivel;
- o alerta ainda nao foi enviado para aquele evento.

Quando alertas sao apagados:

- a tarefa/request correspondente deixa de estar ativa;
- o broker ainda tem registro de entrega do alerta;
- o bot consegue deletar a mensagem no chat.

Observacoes:

- o bot precisa permissao adequada para deletar em grupos;
- chats nao mapeados sao ignorados ou geram erro controlado;
- mensagens inbound sao registradas com source `telegram` ;
- usernames podem ser mapeados para slugs de membros.

22. Action Providers

Action providers permitem acoes externas controladas.

Providers atuais:

Provider	O que faz	Quando usar
<code>one</code>	Acoes locais apoiadas por CLI	Integracoes locais e automacao direta
<code>composio</code>	OAuth e contas conectadas hospedadas	Email, CRM e ferramentas SaaS suportadas

Configuracao:

```
/config set action_provider one
/config set action_provider composio
/config set composio_api_key <key>
```

Regra de seguranga:

- acoes mutantes devem passar por request/aprovacao humana, a menos que o usuario tenha optado explicitamente por modo mais amplo;
- secrets nao devem ser escritos em docs, templates ou commits;
- integracoes sao opcionais e nao fazem parte do nucleo obrigatorio.

23. Nex, GBrain E Integracoes Opcionais

Nex, GBrain, Telegram e Composio sao tratados como opcionais.

Regra de produto:

- nao assumir que Nex esta configurado;
- nao fazer Nex ser caminho obrigatorio;
- usar texto neutro quando a funcao nao for especifica de Nex;
- preservar o nucleo local-first mesmo sem integracoes externas.

Open CoDesign Como Companion Visual

Open CoDesign pode ser usado como ferramenta externa de prototipagem visual para o agente `designer` , para fluxos frontend e para criacao rapida de artefatos HTML, Markdown, PDF ou PPTX.

Ele nao faz parte do nucleo obrigatorio da Maestria. A integracao local e apenas um launcher seguro:

```
.\scripts\launch_open_codesign.ps1
```

Na interface web, o mesmo fluxo aparece em:

```
Studio -> Operador -> Open CoDesign -> Abrir
```

Esse botao chama o broker local em:

```
GET  /integrations/open-codesign/status
POST /integrations/open-codesign/launch
```

O que o launcher faz:

1. Procura uma instalacao existente do Open CoDesign.
2. Prepara a pasta local `temp\open-codesign` para handoff de prototipos.
3. Abre o app quando encontrado.
4. Se nao encontrar o app, mostra comandos de instalacao manual e sai sem instalar nada.
5. Quando aberto pelo Studio, grava um registro de auditoria `external_tool_launched` sem alterar agentes, canais ou blueprints.

Variaveis e parametros:

Item	Uso
<code>WUPHF_OPEN_CODESIGN_EXE</code>	Caminho explicito para o executavel do Open CoDesign
<code>-PrototypeDir <path></code>	Pasta alternativa para exports e handoff
<code>-NoOpen</code>	Prepara/verifica sem abrir o aplicativo

Fluxo recomendado:

1. Rode o launcher.
2. Use Ollama ou uma chave descartavel/de baixo limite nos primeiros testes.
3. Exporte o prototipo para `temp\open-codesign`.
4. Porte apenas o resultado revisado para `web/`, `docs/` ou outro destino real do repo.

Limites de seguranca:

- nao importar automaticamente credenciais reais do Codex, Claude ou ChatGPT;
- nao apontar o Open CoDesign para `D:\Repos` inteiro;
- nao tratar exports gerados como codigo pronto sem review;
- nao mutar agentes, canais, blueprints, `company.json` ou `broker-state.json` a partir desse fluxo.

Abrir Modo Desktop A Partir Da Web

Quando a UI esta aberta no navegador comum, o Studio mostra uma opcao para abrir a mesma sessao no shell desktop Electron:

```
Studio -> Operador -> Modo desktop -> Abrir desktop
```

Esse botao chama:

```
GET /integrations/desktop/preview
POST /integrations/desktop/launch
```

`/integrations/desktop/preview` e read-only. Ele lista as superficies `desktop-shell`, `desktop-tray` e `browser-lab`, informa `readiness`, checks faltantes, superficie canonica e sinais como `optional_wrapper`, `web_studio_canonical` e `no_topology_mutation`. O preview nao abre Electron e nao altera topologia.

O launcher usa a pasta local `desktop/`, executa `npm run start` e passa a URL local da web atual via `MAESTRIA_WEB_URL`. A janela desktop reaproveita o broker que ja esta rodando e define `MAESTRIA_DESKTOP_NO_BROKER=1`, evitando iniciar outro escritorio por engano.

Visibilidade e limites:

- o botao so aparece na web; dentro do desktop ele fica oculto;
- a URL aceita precisa ser local (`localhost`, `127.0.0.1` ou `::1`);
- quando aberto pelo Studio, grava auditoria `external_tool_launched` com origem `dunderia-desktop`;
- nao altera agentes, canais, blueprints, `company.json` ou `broker-state.json`;
- requer dependencias do diretório `desktop/` instaladas, incluindo Electron.

24. Studio E Dev Console

Studio agrupa ferramentas de operacao e bootstrap.

Superficies comuns:

- pacote de bootstrap da operacao;
- geracao de pacote;
- execucao de workflow;
- contexto ativo de canais, tarefas, requests e mensagens;
- snapshot de ambiente;

- blockers;
- acoes disponiveis;
- dev console.

Endpoints relacionados usados pela UI:

```
/operations/bootstrap-package
/studio/generate-package
/studio/run-workflow
/studio/dev-console
/studio/dev-console/action
/operator/overview
/operator/alerts
/agent-sessions
/execution-trace
```

Quando usar:

- inspecionar o estado operacional;
- preparar ou revisar bootstrap;
- executar workflow guiado;
- diagnosticar bloqueios de runtime.

Alertas Do Operador

`/operator/alerts` consolida sinais operacionais em uma fila curta e read-only. Ele nao resolve nem aplica correcoes automaticamente.

O retorno inclui:

- `persisted:false` ;
- `status` geral (`ok` , `degraded` ou `blocked`);
- `summary` por severidade e origem;
- alertas com `severity` , `source` , `title` , `summary` , canal, item relacionado e acao sugerida.

Fontes usadas:

- Runtime Doctor;
- blockers do Studio;
- requests humanos bloqueantes;
- tarefas sem evidencia de resultado obrigatoria;
- limites de execucao esgotados ou pausados;
- locks expirados ou com heartbeat antigo;
- liveness de tarefa que indica falha, resposta vazia, plano sem progresso ou follow-up necessario;
- volume alto de uso/tokens.

Na UI, esses alertas aparecem em:

```
Studio -> Operador -> Operacao -> Alertas
```

Eles servem para decidir o proximo movimento sem abrir todas as telas diagnosticas. Qualquer acao continua passando pelos endpoints e confirmacoes existentes.

Budget / Context Preview

`/budget/context-preview` mostra, sem gravar nada, quais tarefas chegariam perto de limites de tentativa, runtime, custo ou carga de contexto antes de qualquer enforcement novo.

Endpoint:

```
GET /budget/context-preview?channel=general&viewer_slug=human
GET /budget/context-preview?task_id=task-123&viewer_slug=human
```

O retorno inclui:

- `persisted:false`;
- `status geral ( ok , warning ou blocked )`;
- resumo de uso global (`total_tokens` , `session_tokens` , `requests` , `cost_usd` , agentes com uso);
- itens por tarefa com `budget_state` , `context_state` , `would_warn` , `would_block` e `reasons` ;
- metricas de `attempts` , `runtime` e `cost` quando a tarefa tem limites configurados;
- estimativa de contexto por mensagens recentes, artefatos, planos e liveness.

Quando usado:

- revisar tarefas antes de acordar um agente novamente;
- decidir se os limites devem ser ajustados no detalhe da tarefa;
- enxergar tarefas sem budget explicito;
- transformar limites em governanca observavel antes de criar bloqueios mais rigidos.

Session Inspector E Liveness

O Studio usa `/agent-sessions` e `/execution-trace` para inspecionar o estado de execucao sem ler logs brutos.

`/agent-sessions` mostra, por agente e canal:

- status atual;
- `normalized_status` provider-independent (`idle` , `waiting` , `working` , `input_needed` , `completed` , `failed` , `interrupted` , `stale` ou `blocked`);
- atividade e detalhe recente;
- tarefa atual;
- workspace;
- heartbeat;
- uso/tokens quando disponivel;
- proxima acao sugerida;
- ultimo liveness;
- `liveness_history` recente da tarefa atual.

`/execution-trace` inclui `normalized_status` no trace e em cada passo, alem de passos `liveness` quando o runtime registra progresso, bloqueio, falha, resposta vazia ou plano sem avanco duravel. Esses passos sao gravados como acoes auditaveis `liveness_recorded` e aparecem na timeline da tarefa.

`/tasks` também projeta `liveness_state`, `liveness_reason`, `liveness_at` e até cinco eventos em `liveness_history` para cada tarefa com registro de liveness. O detalhe da tarefa no Studio mostra esse histórico diretamente, sem criar estado novo e sem alterar tarefas.

Na UI, a visão curta aparece em:

```
Studio -> Operador -> Operacao -> Session inspector
```

Quando usado:

- entender se um agente está trabalhando, aguardando, bloqueado ou sem progresso durável;
- revisar a sequência de liveness antes de aceitar uma conclusão;
- decidir se deve acordar o agente, pedir evidência, responder pedido humano ou abrir o trace completo.

25. Browser Lab

Browser Lab é uma superfície para trabalho de navegador quando disponível na UI. Ele serve para experimentos, verificações visuais e fluxos que dependem de browser.

Quando uma verificação de browser for relevante para uma tarefa, registre a inspeção como artefato `browser_inspection` no detalhe da tarefa em vez de deixar URL, seletor e screenshot apenas no texto do canal.

O Studio Diagnóstico também consome `/browser/inspection-handoff-preview` para mostrar quais evidências de browser estão prontas para handoff e quais precisam de URL, seletor, texto, viewport ou screenshot antes de orientar um agente. O painel de diagnóstico também mostra previews de sandbox remoto, revisões de scheduler, editor wiki, compatibilidade de providers, widgets de projeto, handoff de arquivos, Desktop/IDE e multi-company para manter esses desenhos visíveis sem habilitar execução remota, install commands, edição de wiki, leitura de arquivos, shell obrigatório ou mutação de topologia.

Os endpoints adicionais dessa visão são:

```
GET /providers/compatibility-preview
GET /studio/project-overview-preview
GET /files/context-handoff-preview?channel=general&viewer_slug=human
```

Eles retornam `persisted:false` e mantêm `mutation_enabled:false` ou `content_read_enabled:false` quando aplicável.

Quando usar:

- testar UI local;
- verificar páginas;
- apoiar tarefas que pedem interação em navegador;
- investigar comportamentos visuais.

26. Configuracao E Secrets

Configurações podem vir de:

- flags da CLI;
- `~/.wuphf/config.json` ;

- secret store local;
- variaveis de ambiente;
- configuracao feita pela UI;
- estado local do escritorio.

Secrets comuns:

- chaves de providers de IA;
- token do Telegram;
- chave Composio;
- credenciais ADC Google quando aplicavel.

Boas praticas:

- nunca commitar secrets;
- nao copiar snapshots privados para templates publicos;
- usar `templates/starter-kit/` como material bootstrap publico;
- tratar `dunderia-public-export/templates/local-runtime-profile` como referencia, nao como instalacao direta;
- usar `docs/local-runtime-profile.md` antes de copiar qualquer material de runtime exportado.

27. Seguranca E Permissoes

Principios:

- local-first por padrao;
- integracoes opt-in;
- ferramentas mutantes com aprovacao humana;
- escopo reduzido de MCP;
- worktrees isoladas;
- topologia protegida;
- secrets fora do repo;
- comandos destrutivos exigem confirmacao explicita.

Topologia protegida inclui:

- `company.json` ;
- `broker-state.json` ;
- onboarding/bootstrap state;
- workflows salvos que recriam agentes/canais;
- seeds ou blueprints que alteram roster ou lista de canais;
- criacao, remocao, renomeacao, reordenacao, reatribuicao ou reconfiguracao de agentes e canais.

Quando uma mudanca envolve topologia, pare e peca autorizacao explicita do usuario na conversa atual.

Para auditoria e recuperacao, trate `company.json` como a fonte de verdade da topologia ativa. `broker-state.json` pode conter historico e registros operacionais de execucoes anteriores, mas reinicios do broker reconciliam o estado ativo para nao ressuscitar agentes, canais ou DMs fora do manifesto atual.

A reconciliacao tambem filtra scheduler, watchdogs, decisoes e memoria privada de canais. Isso evita que follow-ups antigos, snapshots de recuperacao ou memoria de canais removidos recriem canais que ja nao estao no manifesto.

Quando o escritorio esta vazio, os agendadores globais de auditoria de publicacao GitHub, worktrees de tarefas e follow-up do CEO nao recriam itens de agenda. Se um desses jobs existir num estado limpo, o broker remove o registro para manter a fila realmente vazia.

Um `task-worktree-audit` marcado como `cancelled` e tratado como cancelamento explicito do operador. O watchdog preserva esse estado e nao reativa automaticamente o job enquanto o registro cancelado permanecer no scheduler.

28. Assets E Branding

Branding publico atual:

- nome: Maestria;
- logo SVG em `brand/maestria-logo.svg` ;
- logo invertido em `brand/maestria-logo-inverted.svg` ;
- PNGs em `brand/png/` ;
- hero em `assets/hero.png` ;
- favicon e icones da web em `web/public/` .

O que deve ser atualizado ao mudar branding:

- assets em `brand/` ;
- icones e favicon da UI;
- README e docs;
- textos da interface;
- manifestos web;
- referencias publicas.

Compatibilidade:

- alguns nomes tecnicos historicos podem permanecer quando trocar quebraria pacote, import, script ou automacao.

29. Desenvolvimento Local

Comandos comuns:

```
go test ./...
npm --prefix web run build
npm --prefix web run dev
go build -o wuphf.exe ./cmd/wuphf
```

Quando usar:

- `go test ./...` : antes de concluir mudancas Go amplas;
- `npm --prefix web run build` : antes de concluir mudancas na web;
- `npm --prefix web run dev` : desenvolvimento visual local;
- `go build` : validar binario.

Mudancas apenas documentais normalmente nao exigem build, mas devem ser verificadas por leitura, links e ausencia de marcadores em aberto.

30. Debug E Diagnostico

Primeiros passos:

```
wuphf doctor
wuphf doctor --smoke
wuphf log
```

Problemas comuns:

Sintoma	O que verificar
UI nao conecta	broker em 7890 , web em 7891 , auth/token local
Agente nao responde	provider, focus mode, mencao, runner, task owner
Tarefa ficou parada	status, owner, execution lock, blocker, request pendente
Telegram nao envia	token, canal mapeado, permissoes do bot, grupo descoberto
Telegram nao apaga alerta	permissao de delete, registro de entrega, task ainda ativa
Mensagens em canal errado	slug do canal, source, filtro de mensagens, mapeamento surface
Custo alto	provider, loops de recovery, repeticoes, uso por agente
Worktree inconsistente	leases, task worktree path, auditoria, branch

31. Exemplos De Fluxos

Criar plano e tarefas

```
Humano em #general:
@pm quebre a entrega de alertas Telegram em tarefas pequenas e chame @eng quando estiver pronto.
```

O que acontece:

1. Broker registra a mensagem.
2. Launcher acorda PM.
3. PM responde e pode criar tarefas.
4. Se mencionar engenharia, o broker acorda o agente citado.
5. Tarefas aparecem na inbox.

Pedir implementacao

```
@eng implemente a delecao automatica de alerta Telegram quando a solicitacao for resolvida.
```

O que acontece:

1. Engenharia recebe contexto do canal e da tarefa.
2. Runner trabalha em worktree isolada.
3. Mudancas sao registradas.
4. Testes ou validacoes sao executados.
5. Tarefa vai para review ou done com evidencia.

Responder request

Request: Preciso do token do Telegram.

Resposta: Token salvo no secret store; pode testar agora.

O que acontece:

1. Request recebe resposta.
2. Tarefa deixa de aguardar humano.
3. Agente pode ser acordado para continuar.
4. Se havia alerta Telegram, ele pode ser apagado quando resolvido.

Conectar Telegram

```
/connect
Telegram
Selecionar grupo "Operacao"
Mapear para canal "telegram-operacao"
```

O que acontece:

1. Bot token e validado.
2. Grupos sao descobertos via Telegram.
3. Canal surface `telegram` e criado ou atualizado conforme permissao.
4. Transporte long-polla updates.
5. Mensagens fluem entre Telegram e broker.

Auditar uma tarefa

```
wuphf log task-123
```

O que acontece:

1. CLI le recibos locais.
2. Exibe output da tarefa.
3. Operador verifica o que foi feito e o que falta.

32. Contratos Que Nao Devem Ser Quebrados

Preserve estes contratos:

- core local-first;

- broker push-driven;
- runners frescos por turno;
- MCP escapado;
- worktrees isoladas;
- integracoes opcionais;
- topologia protegida;
- acoes mutantes com aprovacao;
- recovery conservador;
- logs e entregas auditaveis;
- manual atualizado junto com mudancas.
- PDF do manual regenerado quando o Markdown mudar.

33. Checklist Para Alteracoes Futuras

Antes de concluir qualquer mudanca, responda:

- Mudou comportamento de usuario?
- Mudou comando, flag, config ou secret?
- Mudou UI, texto, navegacao, icone ou asset?
- Mudou runtime, provider, broker, task, recovery ou worktree?
- Mudou integracao Telegram, Composio, Nex, GBrain, Open CoDesign ou action provider?
- Mudou seguranca, permissoes ou approval?
- Mudou fluxo de onboarding, blueprint, pack ou template?
- Mudou arquivo que afeta topologia protegida?

Se qualquer resposta for sim, atualize este manual com:

- o que mudou;
- como funciona;
- quando acontece;
- exemplo de uso;
- limites ou cuidados.

Depois de atualizar o Markdown, regenere docs/MANUAL.pdf :

```
.\scripts\render_manual_pdf.ps1
```

Se todas forem nao, declare no handoff que a mudanca nao exigiu atualizacao do manual.